

WATERFALL Vs AGILE – COMPARISON TABLE

Feature	Waterfall Model	Agile Model
Approach	Linear and sequential	Iterative and incremental
Process Flow	Moves step-by-step (Requirements → Design → Development → Testing → Deployment)	Continuous cycles (Plan → Develop → Test → Review → Repeat)
Flexibility	Rigid, changes are difficult once a phase is completed	Highly flexible; changes can be made anytime
Requirements	Must be clearly defined at the beginning	Can evolve throughout the project
Customer Involvement	Low; client reviews mainly at the end	High; client provides continuous feedback in every iteration
Delivery	Final product delivered after full development	Working increments delivered frequently (e.g., every Sprint)
Testing	Done after development is complete	Continuous testing within each iteration
Team Collaboration	Limited; roles are clearly defined and follow a structured hierarchy	High collaboration; cross-functional teams work together
Documentation	Extensive and mandatory at each stage	Minimal documentation; focuses on working software
Cost & Time	Higher cost if changes occur; timeline fixed	Cost-effective for changing requirements; timeline adaptive

Sample Application Workflow: Waterfall vs Agile

1. Waterfall Workflow (Simple Linear Flow)

In Waterfall, each phase must finish completely before the next begins.

Waterfall Workflow for “Task Management App”

1. Requirements Gathering

- Understand features like create task, edit task, mark complete.

2. System Design

- Create architecture diagrams, database design, UI mock-ups.

3. Development

- Build full app (backend + frontend) at once.

ACTIVITY – II

AGILE METHODOLOGY

- Agile helps projects adapt quickly to new change requests, making it ideal for dynamic environments.
- The main goal of Agile is **fast and efficient project completion** through short, iterative cycles.
- Agile achieves speed by **removing unnecessary steps** and focusing only on activities that add value.
- All Agile processes share common traits such as **continuous feedback, teamwork, flexibility, and iterative development**.

Agile Manifesto

The Agile Manifesto is a set of foundational principles for agile software development that emphasizes flexibility, collaboration, and customer satisfaction. It Outlines 4 core Values and 12 Principles guide agile practices.

4 Core Values

- People and communication are more important than tools and processes.
- Working software matters more than having lots of documents.
- It is better to work closely with customers than just relying on contracts.
- Being able to change the plan is better than strictly following a fixed plan.

12 Principles of Agile Manifesto

This Agile Manifesto is supported by 12 principles that help to implement the values.

- Make customers happy by delivering useful software early and regularly.
- Accept changes in requirements at any time so the product stays useful and relevant.
- Deliver working parts of the software often, so feedback comes quickly.
- Business people and developers should work together every day for better understanding.
- Build projects with motivated people and give them the support and trust they need.
- Talking face-to-face is the best way to communicate and avoid confusion.
- Working software is the main sign that the project is progressing well.
- Work at a pace that the team can maintain consistently without burning out.
- Focus on good design and technical quality to make the software easy to maintain.
- Keep things simple and avoid doing unnecessary work.
- Let teams organize themselves—they produce better ideas and solutions.
- Regularly review how the team is working and improve whenever needed.

Kanban

Basic Principles

- Show the work visually so everyone can see what's happening.
- Limit how many tasks are being worked on at the same time.
- Manage the flow of work so it moves smoothly without delays.

ACTIVITY – III

AGILE / SCRUM TOOLS

Jira – (Used for tracking work)

Jira is a tool that helps Agile teams plan sprints, manage backlogs, and track tasks on Scrum or Kanban boards. It makes it easy to see what the team is working on and how the project is progressing

Example:

A development team is building a mobile app.

- They create user stories like “Add login screen” or “Fix payment bug” in Jira.
- During sprint planning, they drag these tasks into the *Sprint Backlog*.
- Team members update tasks from *To Do* → *In Progress* → *Done* as they work.

ClickUp – (All-in-one project management tool)

ClickUp is used to organize tasks, create sprint boards, set goals, and document work. It supports Agile teams by keeping everything—tasks, docs, and timelines—in one place, making work easier to manage.

Example:

A marketing team planning a campaign uses ClickUp to:

- Create tasks like “Design poster”, “Write content”, “Schedule posts”.
- Track progress on a sprint board.
- Store campaign documents in the same workspace.

Trello – (Simple board-based tool)

Trello uses boards, lists, and cards to show work visually. It is simple to use and perfect for smaller Agile teams to track to-dos, in-progress work, and completed tasks.

Example:

A school project team uses Trello to organize tasks:

- *To Do*: “Prepare slides”, “Collect data”
 - *Doing*: “Write report”
 - *Done*: “Topic selection”
- This makes it easy for everyone to see what’s happening.

Miro – (Online whiteboard for collaboration)

Miro is a digital whiteboard that Agile teams use for brainstorming, creating sprint plans, user journey maps, and retrospectives. It helps teams collaborate visually during meetings.

Example:

A Scrum team runs a sprint retrospective on Miro:

- They use sticky notes to write “What went well” and “What needs improvement”.
- They vote on the most important issues.
- They create action items for the next sprint.

Asana – (Task and workflow management)

Asana helps Agile teams create tasks, set deadlines, follow timelines, and manage workflows. It gives a clear view of what work is assigned, what’s in progress, and what’s completed.

ACTIVITY – IV

White Box Testing

White box testing means the tester **knows the internal code**, logic, and structure of the application.

- Tester needs programming knowledge.
- Internal logic, loops, and conditions are tested.
- Test cases are based on **code structure**.
- Used mostly by **developers**.
- Ensures code quality, flow, and paths are correct.

Advantages

- Helps find hidden coding errors.
- Optimization of code is possible.
- Covers all logical paths.
- Detects issues early in development.

Disadvantages

- Requires knowledge of code (not easy for all testers).
- Time-consuming.
- Not suitable for testing UI/UX.
- Complex for large applications.

Black Box Testing (Functional Testing)

Black box testing means the tester **does NOT know the internal code**, only tests the application from the **user's point of view**.

- Tester focuses on **input and output**, not code.
- No programming knowledge required.
- Functional behavior is tested.
- Real-world scenarios are checked.
- Most common testing done by QA teams.

Advantages

- Easy to perform, no coding skills needed.
- Tests application exactly like end users.
- Works well for large systems.
- Helps find missing requirements.

Disadvantages

- Cannot test code logic.
- Coverage may be low because internal code is hidden.
- Hard to design test cases without clear requirements.
- Not good for performance of internal components.

ACTIVITY – V

A TEST CASE DESIGN TABLE FOR ATM WITHDRAWAL

A Test Case Design Table with all requested columns.

A Test Case Execution Table

1. Six test cases, each showcasing a different **test design technique**:

- EP = Equivalence Partitioning
- BVA = Boundary Value Analysis
- DT = Decision Table
- ST = State Transition
- UC = Use-Case/Scenario
- EG = Error Guessing (incl. security/negative)

Assumptions (for clarity):

- Daily withdrawal limit = ₹20,000; Per-transaction min = ₹100; max = ₹10,000; step = ₹100 denominations
- Account balance, bank charges, and ATM cash availability are validated by the system
- Valid 4-digit PIN; 3 consecutive wrong PINs → card blocked for the day
- Sufficient notes available (unless specified)

Test Case Design Table

TC No.	Module	Test Case Description	Preconditions	Test Steps	Test Data	Expected Result	Status
TC-01 (EP)	ATM Withdrawal	Withdraw a valid amount within limits with sufficient balance	Card active; PIN known; Account balance $\geq$ ₹5,000; Daily usage ₹0	1) Insert card 2) Select "Withdraw" 3) Enter amount 4) Confirm 5) Collect cash & receipt	Amount = ₹5,000	Cash ₹5,000 dispensed; Balance debited; Receipt printed; Daily limit updated; Transaction success message	Not Run
TC-02 (BVA)	ATM Withdrawal	Withdraw minimum allowed amount	Card active; Balance $\geq$ ₹100; Daily usage ₹0	1) Insert card 2) Withdraw 3) Enter amount 4) Confirm	Amount = ₹100 (min boundary)	Cash ₹100 dispensed; Success; Daily limit reduces to ₹19,900	Not Run
TC-03 (BVA)	ATM Withdrawal	Withdraw just above daily remaining	Card active; Daily used = ₹19,900;	1) Insert card 2) Withdraw 3) Enter	Amount = ₹200	Error shown: "Exceeds daily limit"; No cash; No	Not Run